Inventory & Order Management API

Done by: Mariya Nabhan Saif Allamki

Graduate Technical Assignment

1. Project Overview

Objective

Develop a Spring Boot REST API for managing an online store's inventory and customer orders.

The system supports:

- Category management
- Product management
- Customer management
- Order lifecycle management
- Inventory control
- Low-stock reporting

The application prevents overselling and ensures inventory consistency by updating stock only when orders are confirmed.

2. Technology Stack

Component	Technology
Language	Java 17
Framework	Spring Boot 3
ORM	Spring Data JPA
Database	H2 Database (File Mode)
Build Tool	Maven
Validation	Jakarta Validation
Utility Library	Lombok
API Style	REST

3. Layered Architecture

The application follows a layered architecture:

```
Controller Layer
        ↓
Service Layer
        ↓
Repository Layer
        ↓
Database
```

Responsibilities:

Controller

- Handles HTTP requests.
- Delegates work to services.

Service

Contains business logic:

- Order confirmation
- Inventory updates
- Status transitions
- Validation of business rules

Repository

Responsible for database interaction using Spring Data JPA.

Entity

Represents database tables.

DTO

Used for request and response objects.

4. Package Structure

```
com.mariya.inventory
│
├── category
│
├── product
│
├── customer
│
├── order
│
├── exception
│
└── InventoryApplication
```

Each module contains:

```
controller
dto
entity
repository
service
```

5. Database Design

The system consists of five tables:

Categories

Stores product categories.

Products

Stores products and inventory quantities.

Customers

Stores customer information.

Customer Orders

Stores order information and lifecycle timestamps.

Order Items

Stores products belonging to orders.

6. Entity Attributes

Category

- Category_id

- Category_name

Product

-Product_id

- Product_name

- sku

- Product_price

- Stock_ quantity

Customer

- Customer_id

- Customer_name

- Customer_email

Order

- Order_id

- status

- created_at

- confirmed_at

- fulfilledAt

- Total_amount

- Shipped_at

- Cancelled_at

- Delivered_at

OrderItem

- Order_item_id

- quantity

- unitPrice

- Line_total

7. Database Access

Access the database

Run the application and open:

http://localhost:8080/h2-console

Use:

JDBC URL: jdbc:h2:file:./data/inventorydb

User Name: sa

Password: (leave empty)

From there you can execute the SQL commands:

SELECT * FROM CATEGORIES;

SELECT * FROM PRODUCTS;

```
SELECT * FROM CUSTOMER_ORDERS;
SELECT * FROM ORDER_ITEMS;
```

---

8. Test Cases

1. Create category

POST http://localhost:8080/api/categories

Body → raw → JSON:

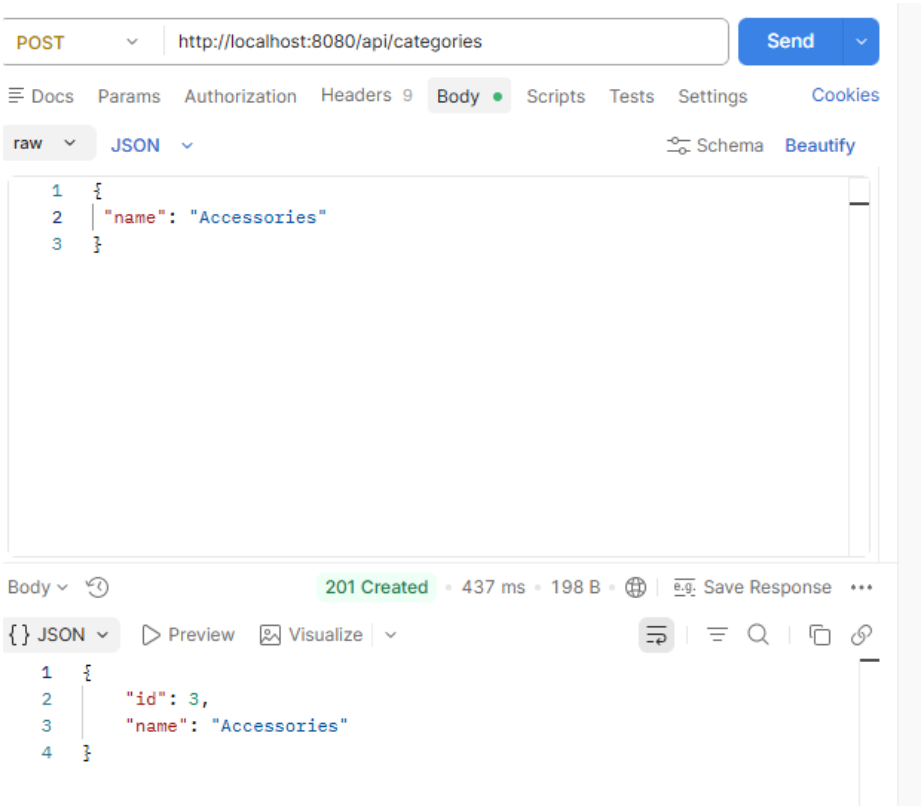
```
{
  "name": "Electronics"
}
```

Expected status:

`201 Created`

Expected response:

```
{
  "id": 1,
  "name": "Electronics"
}
```



2. Create product with stock

POST http://localhost:8080/api/products

{

"name": "Laptop",

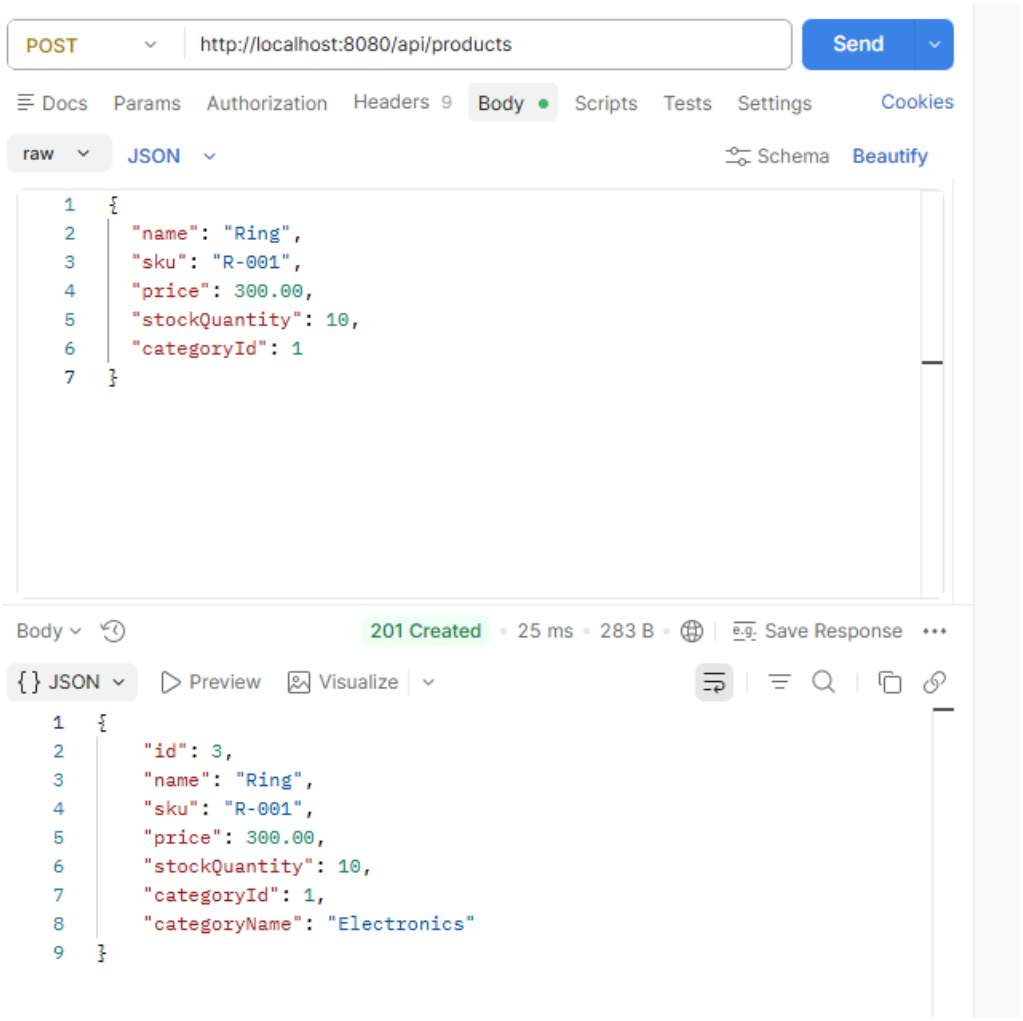
"sku": "LAP-001",

"price": 1200.00,

"stockQuantity": 10,

"categoryId": 1

}



3. Create customer

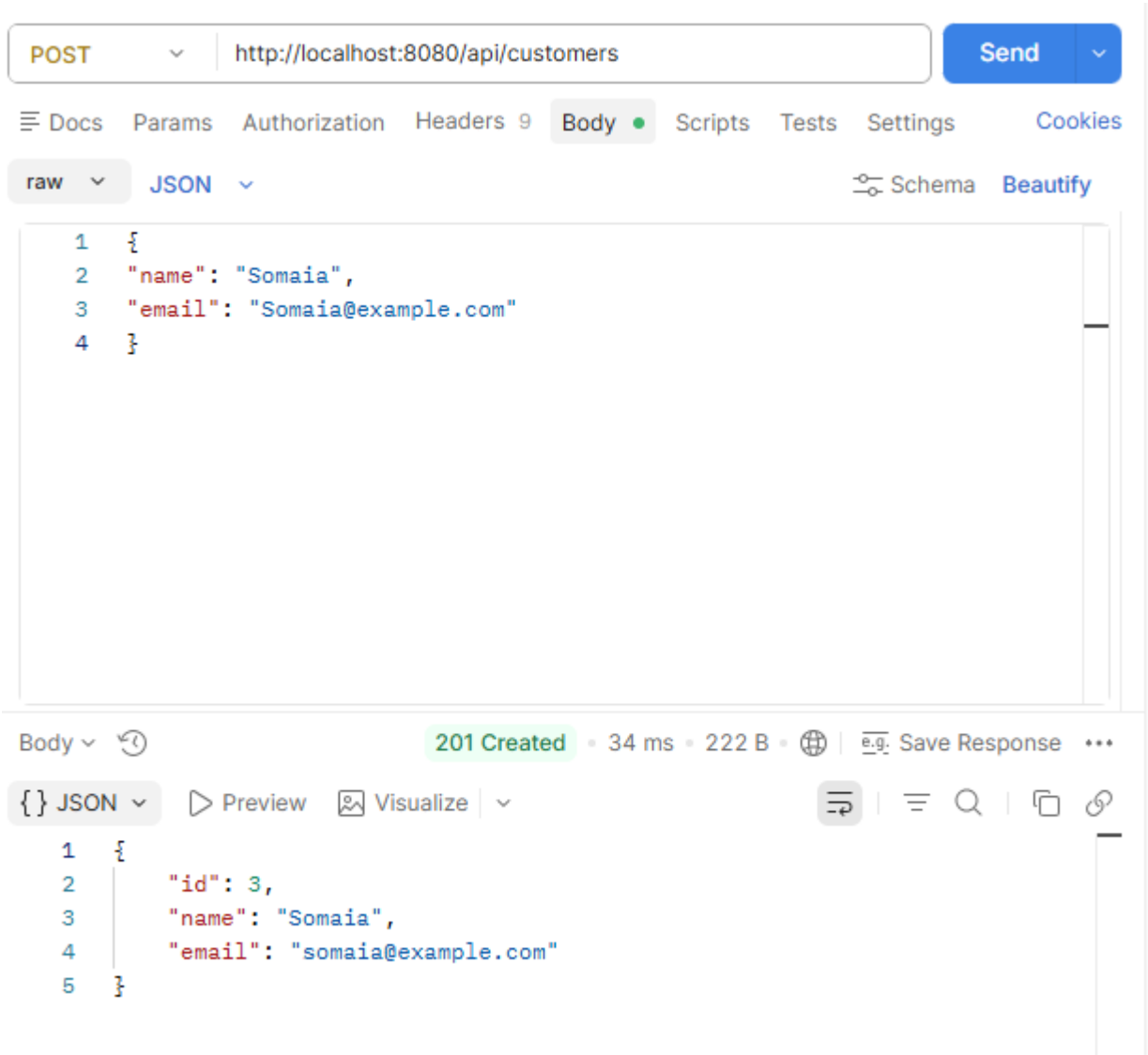
POST http://localhost:8080/api/customers

{

"name": "Fatma",

"email": "fatma@example.com"

}

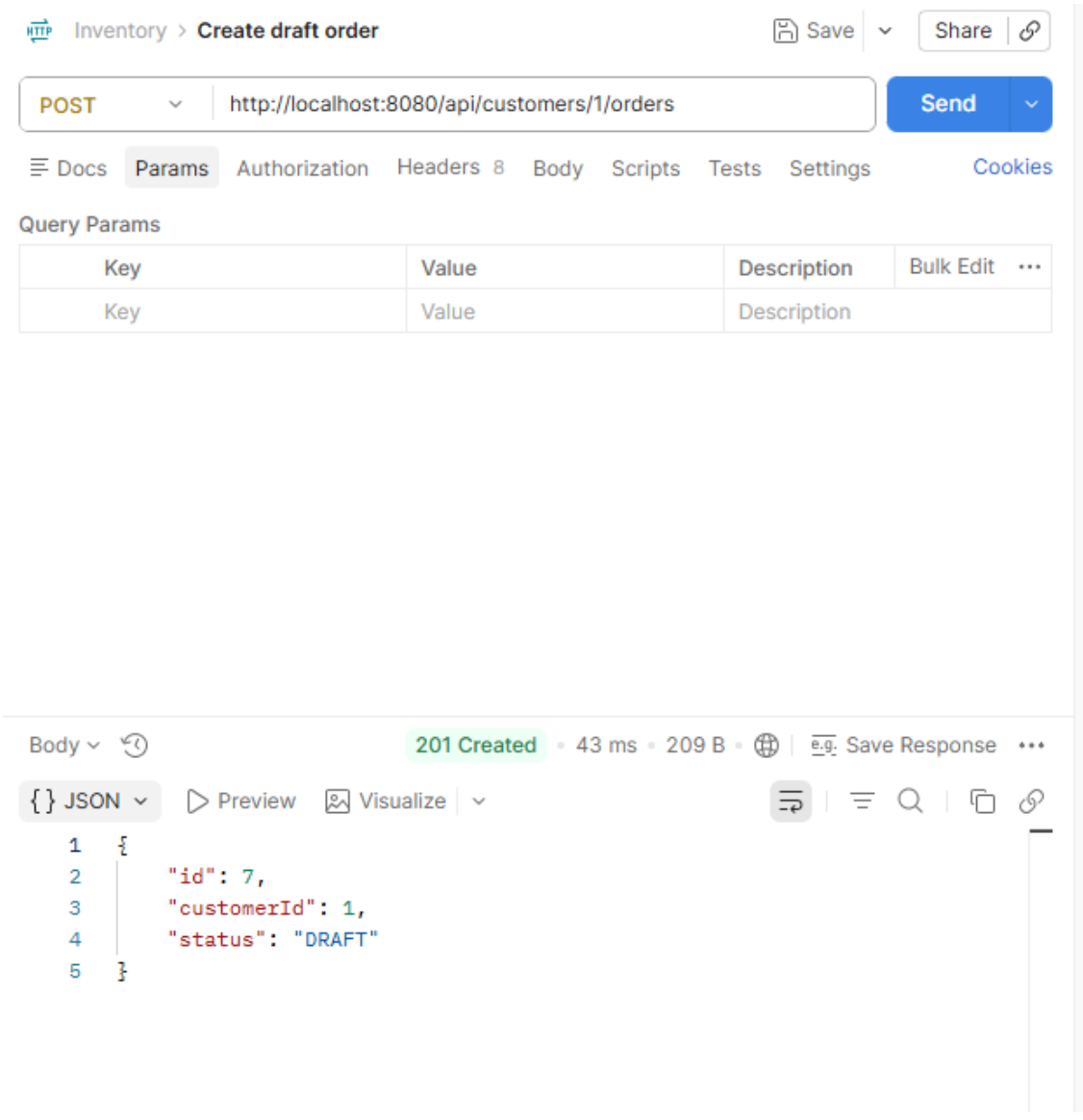


4. Create draft order

POST http://localhost:8080/api/customers/1/orders

Output:


```
{
    "id": 2,
    "customerId": 1,
    "status": "DRAFT"
}
```

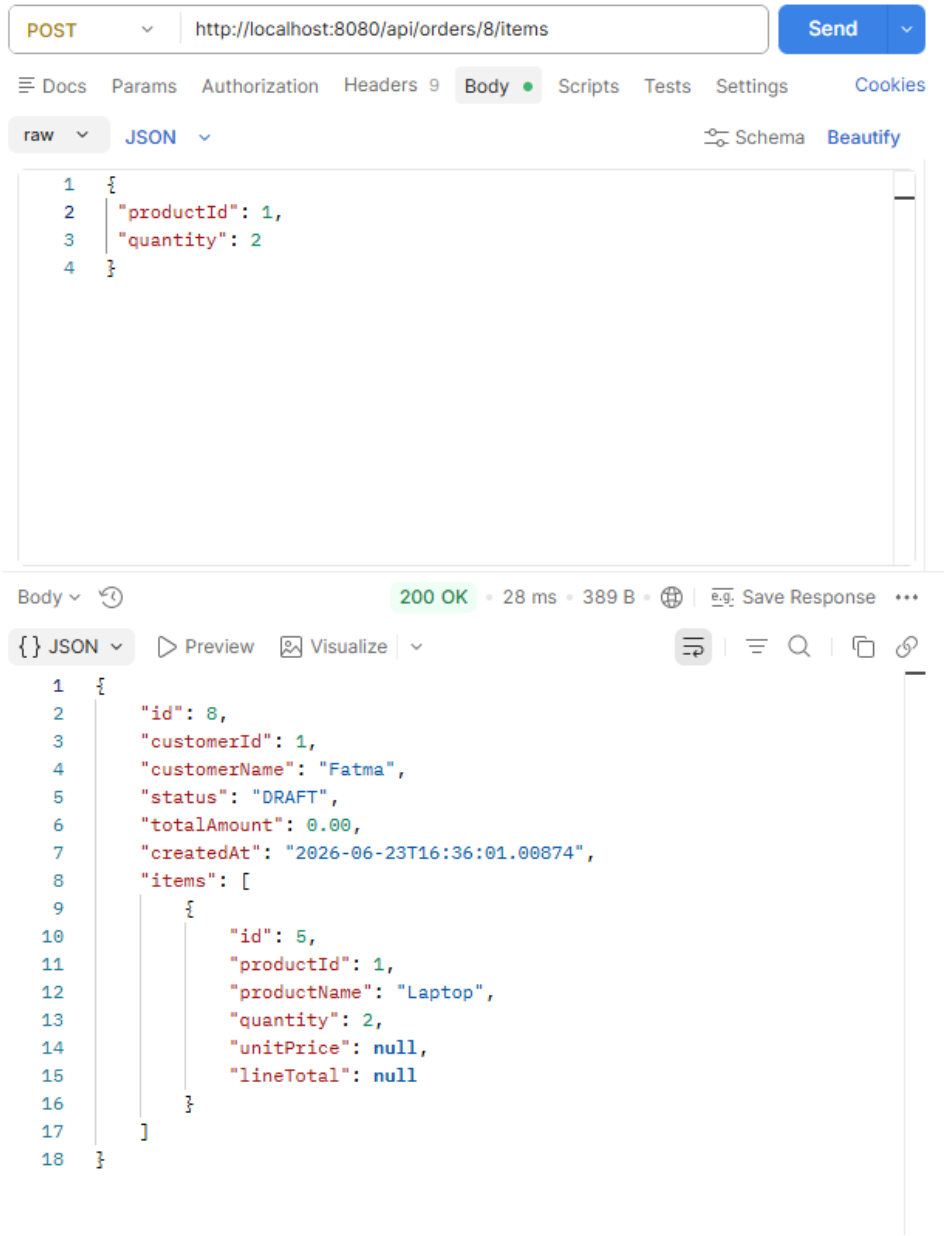



5. Add item to draft order

POST http://localhost:8080/api/orders/1/items

Body:

```
{
  "productId": 1,
  "quantity": 2
}
```

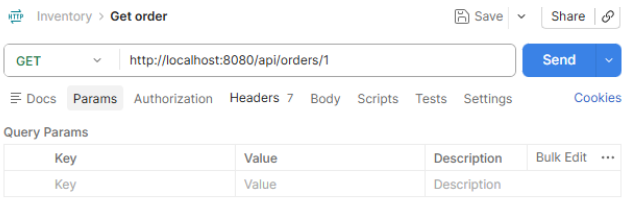


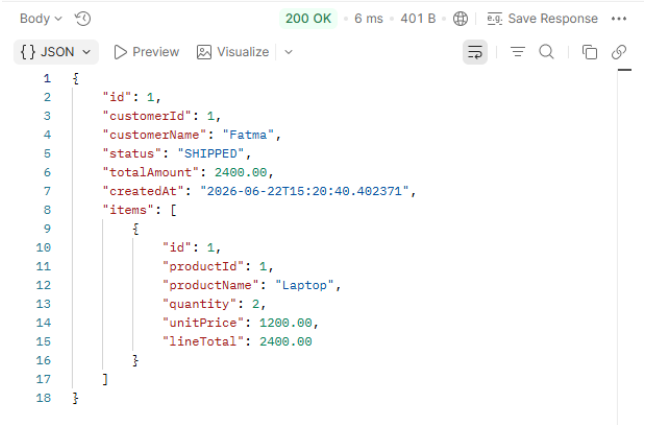
6. Get order

GET http://localhost:8080/api/orders/1

Expected:

```
{
  "id": 1,
  "status": "DRAFT",
  "totalAmount": 0,
  "items": [
    {
      "productId": 1,
      "quantity": 2,
      "unitPrice": null,
      "lineTotal": null
    }
  ]
}
```





7. Confirm order

Graduate Technical Assignment

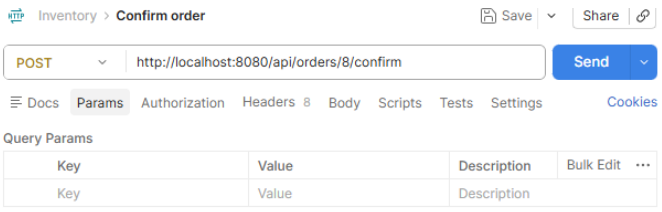
POST http://localhost:8080/api/orders/1/confirm

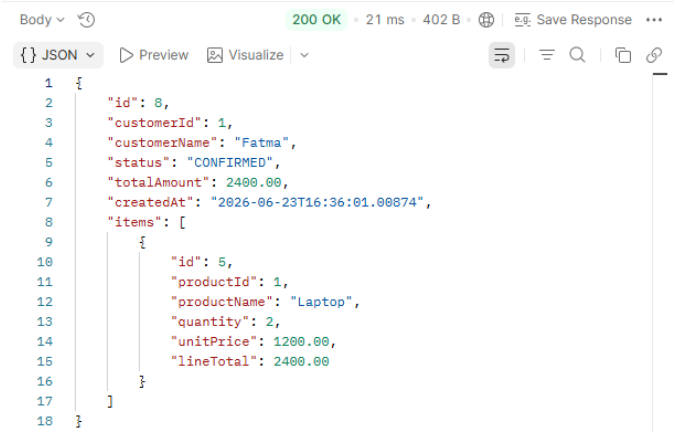
Expected:

```
{
  "id": 1,
  "status": "CONFIRMED",
  "totalAmount": 2400.00
}
```

Item should now have:

```
{
  "unitPrice": 1200.00,
  "lineTotal": 2400.00
}
```



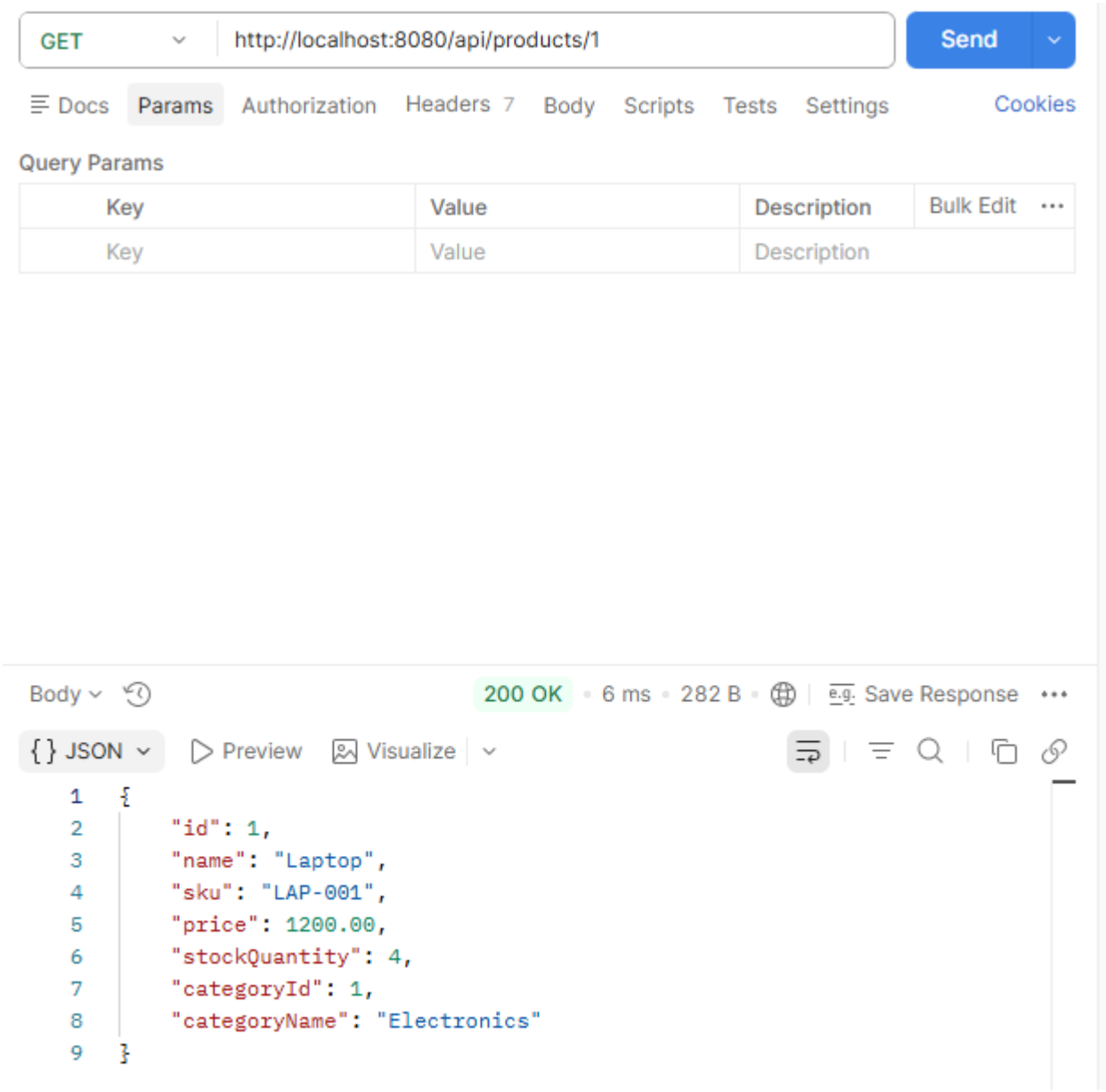


8. Check product stock was deducted

GET http://localhost:8080/api/products/1

Expected:

```
{
  "stockQuantity": 8
}
```



9. Ship order

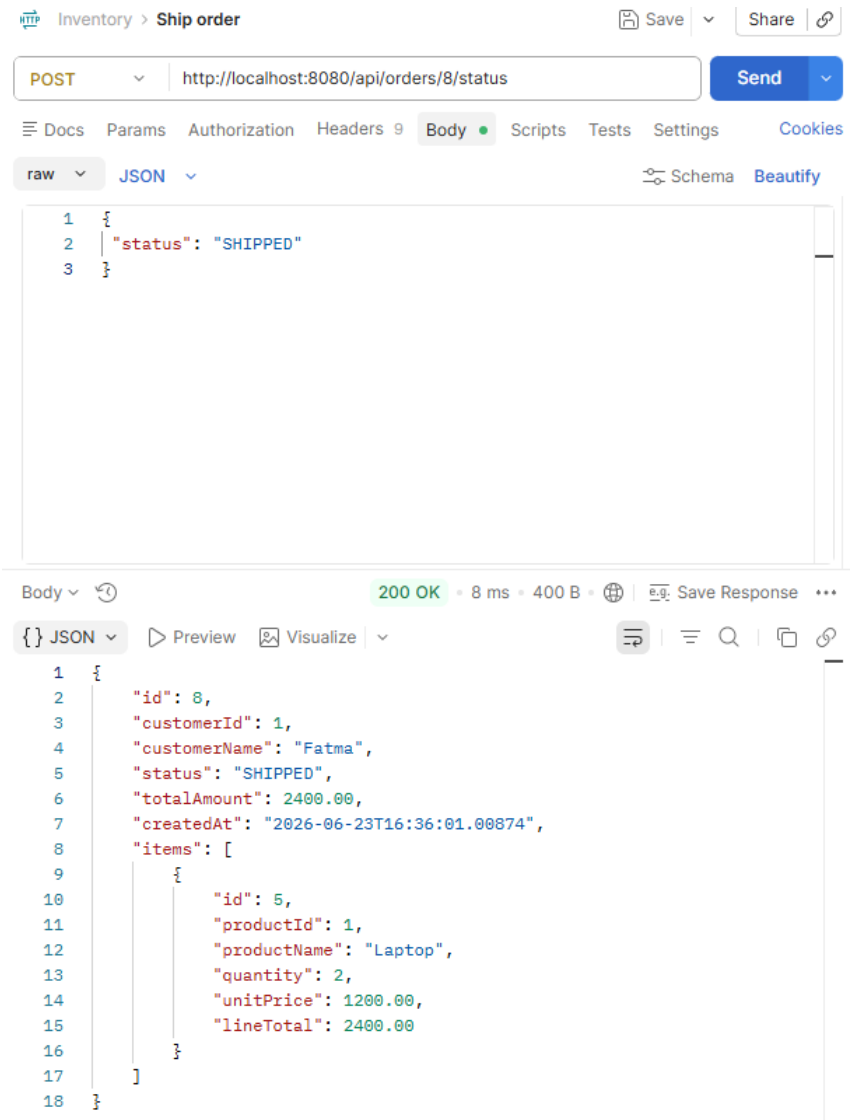
POST http://localhost:8080/api/orders/1/status

Body:
Graduate Technical Assignment

```
{
  "status": "SHIPPED"
}
```

Expected:

```
{
  "status": "SHIPPED"
}
```



10. Deliver order

POST http://localhost:8080/api/orders/1/status

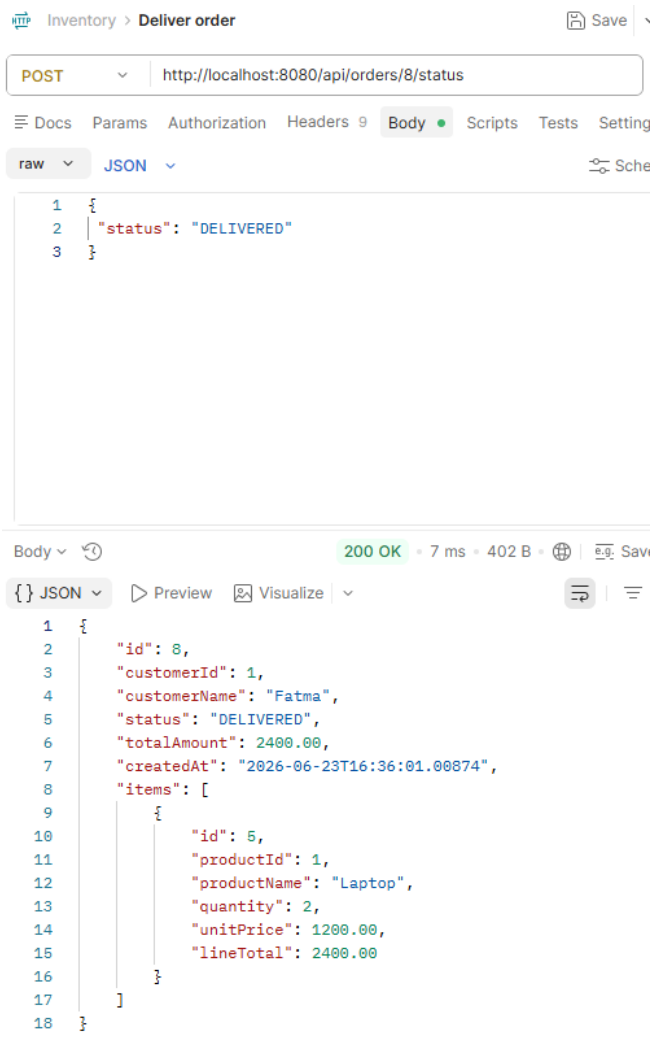
Body:

Graduate Technical Assignment

```
{
  "status": "DELIVERED"
}
```

Expected:

```
{
  "status": "DELIVERED"
}
```



9. Failure cases

A. Confirm empty order

Create a new draft order:

Graduate Technical Assignment

POST http://localhost:8080/api/customer/1/orders

Then confirm it without items:

POST http://localhost:8080/api/customer/2/confirm

Expected error:

```
Cannot confirm an empty order
```

B. Add item after order is confirmed

Use confirm order 1:

POST http://localhost:8080/api/orders/1/items

Body:

```
{
  "productId": 1,
  "quantity": 1
}
```

Expected error:

```
Order must be in DRAFT status
```

C. Confirm order with quantity greater than stock

Create new draft order:

POST http://localhost:8080/api/customers/1/orders

Add item:

POST http://localhost:8080/api/orders/3/items

Body:

```
{
  "productId": 1,
  "quantity": 999
}
```

Confirm:

POST http://localhost:8080/api/orders/3/confirm

Expected error:

```
Insufficient stock for product: Laptop
```

D. Cancel confirmed order and restore stock

Create another order, add quantity 1, then confirm.

POST http://localhost:8080/api/customers/1/orders

POST http://localhost:8080/api/orders/4/items

Body:

```
{
  "productId": 1,
  "quantity": 1
}
```

Confirm:

POST http://localhost:8080/api/orders/4/confirm

```
Check stock decreased, then cancel:
```

POST http://localhost:8080/api/orders/4/status

Body:

```
{
  "status": "CANCELLED"
}
```

Expected:

```
{
  "status": "CANCELLED"
}
```

Then check product again:

GET http://localhost:8080/api/products/1

Stock should be restored by 1.

Test validation error:

POST http://localhost:8080/api/customers

Body:

```
{
 "name": "",
 "email": "wrong-email"
}
```

Expected:

```
{
 "status": 400,
 "error": "Bad Request",
 "message": "Validation failed",
 "validationErrors": {
 "name": "Customer name is required",
```

"email": "Customer email must be valid"

}

}

10. Conclusion

The Inventory & Order Management API provides a layered and normalized solution for managing inventory and customer orders. The design follows object-oriented principles, enforces business rules through the service layer, and guarantees inventory consistency by updating stock only upon order confirmation and restoring stock upon cancellation.